

Final Project Documentation

Course: Basics of Programming II **Course ID:** [Check in Neptun] **Student Name:** [Your Name]
Neptun Code: [Your Neptun Code] **Lab Group ID:** [Your Lab Group ID] **Lab Instructor:** Dr. Dmitriy Dunaev (or [Your Lab Instructor's Name]) **Project Title:** C++ Order Book and Matching Engine Simulation **Project Type:** Final Project **Submission Date:** [Submission Date]

1. Introduction

The objective of this project is to simulate a financial matching engine using an Order Book model. In financial markets, an order book is an electronic list of buy and sell orders for a specific financial instrument, organized by price level.

The problem domain involves receiving a continuous stream of incoming orders (bids and asks) and matching them based on price-time priority. This project implements an event-driven simulation where orders are sequentially read from an input stream, processed by the matching engine logic, and executed. If an order cannot be matched immediately, it is stored in the respective order book for future matching. The system is designed strictly using the C++ Standard Library, applying Object-Oriented Programming (OOP) paradigms, dynamic memory management, exception handling, and file-based I/O.

2. Program Interface

The program is designed to be run as a console application.

- **Compilation:** To set up the environment, the user must compile the source files using a standard C++ compiler (e.g., `g++`). The command is:

```
g++ -o orderBookSim.exe main.cpp order.cpp limit.cpp market.cpp  
orderBook.cpp streamReader.cpp logger.cpp stop.cpp stopL.cpp stopM.cpp
```

- **Execution:** The program can be executed via the command line by running the compiled executable:

```
./orderBookSim.exe
```

- **Termination:** The program terminates automatically once the end of the input stream (the input file) is reached and all data structures have been successfully deallocated. No additional parameters are required to run the simulation, as it defaults to reading from a predefined input file.

3. Program Execution

From an end-user perspective, the execution of the simulation is entirely automated and event-driven.

1. **Input Phase:** Upon startup, the program looks for an input file containing the order stream.

2. **Simulation Phase:** The program parses each line of the input file sequentially. For each order, it determines the ticker symbol (e.g., AAPL, MSFT) and routes the order to the appropriate order book.
3. **Output Phase:** The program produces no direct console output during normal execution unless an error occurs (such as an invalid format in the input file). Instead, all actions (trades, order depletions) are logged silently to output text files. The user can open these log files to review the chronological execution of trades.

4. Input and Output

Input Format

The program expects an input file named `streamIn.txt` located in the same directory as the executable. The file contains a stream of space-separated order properties. **Format:** `[ID] [Timestamp] [Ticker] [Side] [Execution Type] [Quantity] [Price (if limit)]`

Example Input:

```
2023.10.25.09.30.00 AAPL buy limit 100 150.50
2023.10.25.09.30.05 AAPL buy market 25
```

Output Format

The program generates output log files. It generates a global `mainOutputStream.txt` file that records all trades and depletions across all tickers, as well as individual stream files for specific tickers (e.g., `AAPLStream.txt`). **Example Output:**

```
2023.10.25.09.30.05 Trade in: AAPL between: 2 and 1 at price: 151.00 for: 25 stock
Deplete in stock: AAPL, at side: b, at price: 150.50
```

5. Program Structure

The program is structured heavily around OOP principles, utilizing base classes and inheritance to represent the problem domain.

- **order (Base Class):** An abstract representation of a financial order. It holds common properties such as ID, timestamp, ticker, type (buy/sell), execution (market/limit), and quantity. It includes a virtual destructor for safe polymorphic deletion.
- **limit and market (Derived Classes):** These classes inherit from the `order` base class. The `limit` class includes an additional `price` property and overloads the `<` and `>` operators for price comparisons.
- **orderBook:** The core module of the program. It encapsulates both the data structures for holding resting orders and the algorithmic logic for the matching engine. It contains two primary sides: `BuyerSide` and `SellerSide`, which map price levels (`double`) to a vector of active orders. It utilizes `std::priority_queue` to keep track of the best bid (highest buyer) and best ask (lowest seller) in $O(1)$ time. The `executeTrade()` function handles the recursive matching of incoming orders against the resting liquidity based on price-time priority.

- **StreamReader**: Responsible for file I/O operations. It safely reads and parses `streamIn.txt`, handling dynamic memory allocation (`new`) to instantiate the correct derived `order` objects. It implements `try/catch` logic to safely handle corrupted or invalid input lines.
- **logger**: A utility class that manages file output. It dynamically opens and appends trade execution records to the appropriate output files.

Applied Extra C++ Techniques

- **Inheritance**: Base `order` class and derived `limit`, `market`, and `stop` classes.
- **Polymorphism**: Utilization of virtual functions (virtual destructor in the base class) and dynamic casting to safely handle derived objects.
- **Operator Overloading**: The `<` and `>` operators are overloaded within the `limit` class to simplify price comparisons.

6. Testing and Verification

The software was tested by feeding it a mock `streamIn.txt` file containing edge cases. The objectives of the testing were to ensure:

1. **Memory Safety**: Verifying the absence of memory leaks and dangling pointers. Extensive dynamic memory management testing was performed to ensure that orders that are fully matched are deleted, while unfulfilled market orders are properly caught and deallocated without causing `std::bad_alloc` errors.
2. **Algorithmic Correctness**: Verifying that a market order successfully walks up the book (partially filling across multiple price levels) and that time-priority is respected for orders residing at the same price level.
3. **Exception Handling**: Inputting negative quantities or invalid string types into the stream to verify that the `StreamReader` successfully catches and throws exceptions without crashing the program.

7. Improvements and Extensions

While the core matching engine is highly functional, several improvements could be made in future iterations:

- **Data Structure Optimization**: The current implementation uses a `std::vector` inside a `std::map` to track orders at specific price levels, relying on an $O(N)$ `reallocVector` garbage collection method to remove depleted orders. Replacing the vector with a `std::deque` or `std::list` would allow for $O(1)$ removal of fulfilled orders, drastically improving the engine's speed.
- **Recursion Optimization**: The `executeTrade` method uses recursion to sweep through multiple price levels. Converting this to a `while` loop would prevent potential Stack Overflow exceptions during massive market orders.
- **Future Extensions**: The system architecture has already laid the groundwork for `stop` orders (Stop-Loss and Stop-Limit), which will be implemented by adding a secondary "Trigger Queue"

that activates when the last traded price crosses a specific threshold. Future extensions will also include an $O(1)$ order cancellation feature using an `std::unordered_map` to map Order IDs to their memory addresses.

8. Difficulties Encountered

The most significant difficulty encountered during development was managing C++ object lifecycles and dynamic memory. Because orders must exist across multiple scopes (created in the stream reader, processed in the main loop, and stored long-term in the order book), keeping track of pointer ownership was highly complex. Initial iterations suffered from double-free errors and dangling pointers when `main` attempted to delete an order that was still resting in the order book's data structures. Furthermore, discovering the nuances of C++ initialization order (where member variables are initialized based on declaration order in the header, rather than constructor list order) proved challenging but provided highly valuable experience in debugging `std::bad_alloc` and `std::length_error` crashes.

9. Conclusion

The development of this Order Book Simulation successfully bridges the gap between theoretical C++ concepts and a practical, industry-relevant financial engineering problem. The final product successfully meets all core requirements—including OOP design, dynamic memory management, exception handling, and file manipulation—while also implementing advanced concepts like inheritance and operator overloading. The project stands as a robust foundation that can be easily extended into a fully featured algorithmic exchange.

10. References

- Bjarne Stroustrup, *The C++ Programming Language*, 4th ed., Addison-Wesley Professional, 2013.
- cplusplus.com, "C++ Standard Library Documentation", [Online]. Available: <http://www.cplusplus.com/reference/>.

Note: This documentation was generated in accordance with the Basics of Programming II Final Project requirements.